Home Assignment: AI Reliability Assistant

Time budget

Please spend no more than **3 hours**.

We value a clean, focused implementation over a broad one that only works for the example questions. The goal is not to build a perfect assistant, but to show how you think about **GenAI architecture, orchestration, grounding, tool use, evaluation, and production readiness**.

Scenario

You are building a feature for an industrial reliability platform.

Reliability engineers and maintenance planners want to ask natural-language questions and get grounded, explainable answers about:

- machine health
- active alerts
- maintenance history
- open work orders
- what maintenance should do next

Assume the dataset is small for assignment purposes, but the real platform has thousands of machines across many sites.

The platform is multi-tenant. Different users can see different subsets of machines, sites, alerts, and work orders. You do **not** need to implement authorization, but your README should explain where authorization and user-level data scoping would live in your design.

Data

You will receive four JSON files:

- `machines.json`
- `alerts.json`
- `work_orders.json`

- `maintenance_history.json`

Some fields are structured. Some are free text. Both matter.

Your assistant should use only the provided data. It should not invent machine facts, site facts, alerts, work orders, or maintenance recommendations that are not supported by the data.

Required architecture

Your solution should include a lightweight **orchestration/planning layer**.

The orchestrator should:

- interpret the user question
- decide which specialized capabilities are needed
- call those capabilities
- synthesize a final grounded answer

Your solution should include **multiple specialized capabilities**. These may be implemented as agents, tools, classes, services, or functions. We intentionally do not prescribe the exact architecture. Choose the approach you think is appropriate for a 3-hour assignment and explain your tradeoffs in the README.

Examples of possible capabilities:

- reliability / alert analysis
- maintenance / work-order analysis
- site comparison
- data retrieval / filtering
- final answer synthesis

We expect clear separation of responsibilities, grounded outputs, and a trace showing the orchestration decisions and capability calls.

You may use LangChain, LangGraph, plain Python, or any other framework. Framework choice is less important than clear design.

Required questions

Your assistant must support the following questions:

1. **Which machines should maintenance prioritize this week, and why?**
2. **What should we do about machine M-XXX?**

3. **Compare Site A and Site B and recommend where maintenance should focus first.**
4. **Are there any machines with heat- or temperature-related issues we should look at?**

It should also gracefully handle:

- out-of-scope questions
- ambiguous questions, for example references to machines or sites that do not exist
- questions where the data does not support a confident answer

---

Output format

Every answer should be a structured response containing at least:

```
None
{
 "answer": "...",
 "reasoning": "...",
 "dataUsed": ["machines", "alerts", "work_orders", "maintenance_history"],
 "machinesReferenced": ["M-001"],
 "limitationsOrAssumptions": ["..."]
}
```

You may extend this with additional fields such as:

```
None
{
 "confidence": "high | medium | low",
 "capabilitiesUsed": [],
 "retrievedRecords": [],
 "trace": []
```

```
}
```

The exact schema is up to you, but it should be consistent and easy to understand.

Trace requirement

For each request, emit a minimal trace showing:

- what the orchestrator decided
- which capabilities were called
- what data each capability used
- what each capability returned before final synthesis

This can be printed to stdout, written to a log file, or included in the response.

Example:

```
None
{
 "trace": [
   {
     "step": "planner",
     "decision": "Question requires reliability risk
ranking and maintenance workload analysis."
   },
   {
     "step": "reliability_analysis",
     "input": "machines and active alerts",
     "output": "M-101 and M-204 have high-severity active
alerts."
   },
   {
```

```
      "step": "maintenance_analysis",
      "input": "work orders for M-101 and M-204",
      "output": "M-101 has an overdue work order."
    },
    {
      "step": "synthesis",
      "output": "Prioritize M-101 first, then M-204."
    }
  ]
}
```

Requirements

Build a runnable Python application — CLI or HTTP API, your choice.

It should:

- accept a natural-language question
- return a grounded structured answer
- use only the provided data
- refuse or hedge when the data does not support a confident answer
- include a clear orchestration/planning layer
- include multiple specialized capabilities
- emit a minimal trace for each request
- include a small evaluation harness with 3–5 cases

Each evaluation case should include:

- a question
- the key facts the answer must contain
- expected machines/sites referenced, where relevant

Running the evaluation should print pass/fail per case.

Example:

```
None
python eval.py
```

Example output:

```
None
[PASS] prioritize_this_week

[PASS] temperature_related_issues

[FAIL] compare_sites_missing_fact
```

LLM usage

- Make sure answers remain grounded in the provided data.
- Include safeguards against hallucination

README

Please cover:

1. How to run the application.
2. How to configure the LLM API key.
3. A short architecture overview.
4. How the orchestration/planning layer works.
5. What specialized capabilities exist and what each one is responsible for.
6. How your design would handle 10,000 machines instead of the small sample.
7. Where authorization and tenant/user-level data scoping would live in a production deployment.
8. How you prevent or reduce hallucinations.
9. How tracing/evaluation would evolve in production.
10. Key tradeoffs you made under the 3-hour time budget.
11. What you would improve with more time.

Submission

Please submit:

- source code

- README
- setup instructions
- a few example questions and outputs

Your examples should include:

- one supported question
- one out-of-scope question
- one question where the data does not support a confident answer